



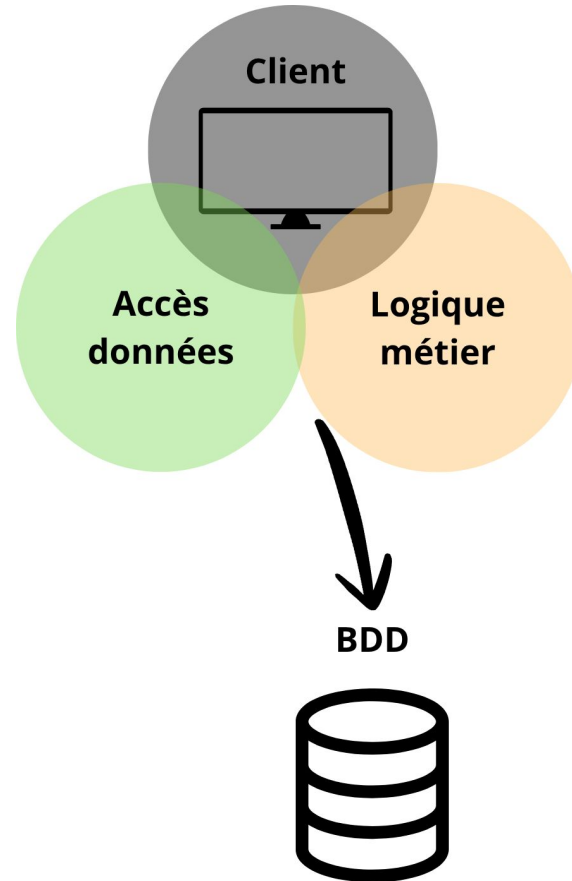
AL

Formation - Du Code à l'Architecture

Architecture monolithique

L'architecture monolithique

- **monolithe** : unité logicielle unique où l'interface utilisateur, la logique métier et l'accès aux données sont regroupés dans un seul artefact de déploiement



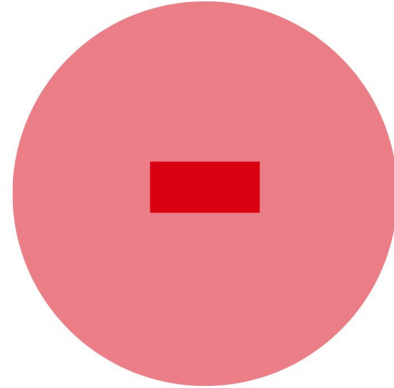
Les points forts d'une architecture monolithique

- Excelle pour les petites équipes et MVP (latence réseau nulle entre les composants)
- Mise en place immédiate, débogage simple
- Simplicité opérationnelle



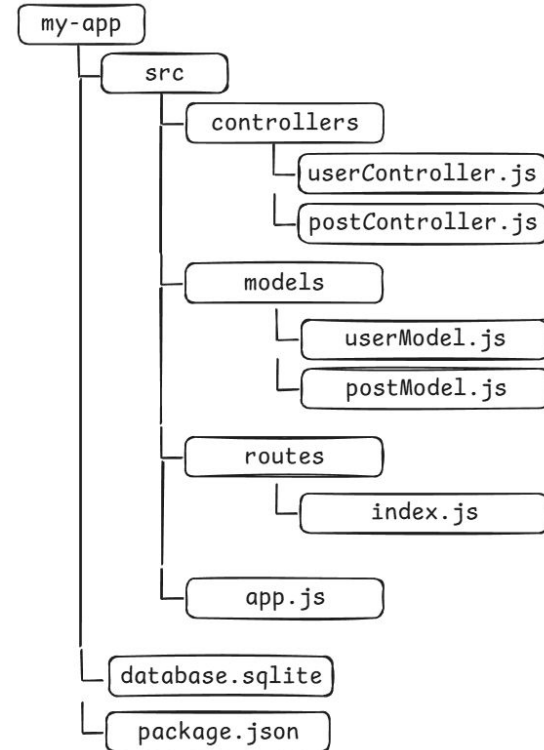
Limite du monolithique

- L'accumulation de code mène au couplage fort
- Rend les mises à jour risquées et le scaling coûteux.



Exemple : Node JS (monolithique classique)

- Dans ce modèle, le code est organisé par type technique
- Les domaines sont mélangés
- On peut également tout retrouver dans un seul fichier



Exemple

```
app.js - 1

const express = require("express");
const Database = require("better-sqlite3");

const app = express();
app.use(express.json());

const db = new Database("database.db");

// Base de données et ajout de données de test (option)
// db.exec ... CREATE et db.prepare ... INSERT
```

```
app.js - 2

app.get("/items", (req, res) => {
  try {
    const items = db.prepare('SELECT * FROM items').all();
    res.json(items);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

app.post("/buy", (req, res) => {
  const { playerId, itemId } = req.body;
  if (!playerId || !itemId) {
    return res.status(400).json({ error: "playerId et itemId sont requis" });
  }

  try {
    // Utilisation d'une transaction pour s'assurer que les deux mises à jour réussissent
    ensemble
    const processPurchase = db.transaction((pId, iId) => {
      const player = db.prepare('SELECT * FROM players WHERE id = ?').get(pId);
      const item = db.prepare('SELECT * FROM items WHERE id = ?').get(iId);

      if (!player) throw new Error("Joueur non trouvé");
      if (!item) throw new Error("Objet non trouvé");

      if (item.stock <= 0) throw new Error("Objet en rupture de stock");
      if (player.gems < item.price) throw new Error("Fonds insuffisants");

      // Mise à jour des stocks et de l'argent
      db.prepare('UPDATE items SET stock = stock - 1 WHERE id = ?').run(iId);
      db.prepare('UPDATE players SET gems = gems - ? WHERE id = ?').run(item.price, pId);

      return {
        success: true,
        message: "Achat réussi",
        remainingGems: player.gems - item.price,
        remainingStock: item.stock - 1
      };
    });

    const result = processPurchase(playerId, itemId);
    res.json(result);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});
```

Exemple suite

```
app.js - 3

app.post("/more-gemmes", (req, res) => {
  const { playerId, amount } = req.body;

  if (!playerId || amount === undefined) {
    return res.status(400).json({ error: "playerId et amount sont requis" });
  }

  if (amount <= 0) {
    return res.status(400).json({ error: "Le montant doit être supérieur à 0" });
  }

  try {
    const info = db.prepare('UPDATE players SET gems = gems + ? WHERE id = ?').run(amount,
playerId);
    if (info.changes === 0) {
      return res.status(404).json({ error: "Joueur non trouvé" });
    }

    const player = db.prepare('SELECT * FROM players WHERE id = ?').get(playerId);
    res.json({ success: true, message: `${amount} gemmes ajoutées`, newBalance: player.gems
});
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

app.listen(3000, () => {
  console.log("Server is running on port 3000");
});
```

Exercise

Ex 1 - Monolithique spaghetti

- Créer une API Express pour une cantine :
 - Utilisateurs : des étudiants qui ont un solde (argent).
 - Menu : Une liste de plats avec un prix et un stock.
 - Commandes : Quand un étudiant achète un plat, on vérifie son solde, on décrémente le stock, et on enregistre la transaction
- Suivre les contraintes suivantes :
 - Tout dans app.js ou index.js
 - La logique est directement dans les routes
 - Ni service ni contrôleur
 - La route commande manipule l'objet utilisateur et menu
 - GET /menu voir les plats et stock
 - POST /order : Vérifier le stock, vérifier le solde de l'étudiant, déduire l'argent et réduire le stock
 - POST /wallet/add : Ajouter de l'argent sur le compte.

Vers la modularité

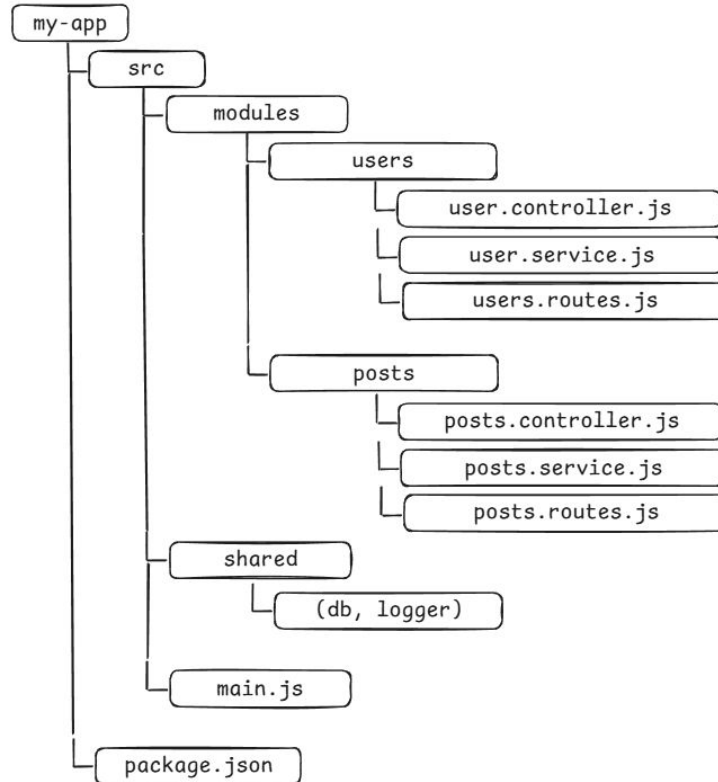
Le Monolithe Modulaire

- Approche intermédiaire où le code est strictement séparé en modules internes
- Ici, on organise le code par **domaine métier**
- Chaque module est une “mini-application isolée”

Avantages versus inconvénients

- **Organisation** : Code rangé par métier plutôt que par type de fichier.
- **Découplage** : Moins d'effets de bord lors des modifications.
- **Évolutif** : Transition facile vers les microservices si besoin.
- **Clarté** : Meilleure compréhension du projet pour les nouveaux arrivants.
- **Rigueur** : Risque de dériver vers un monolithe classique sans outils de contrôle.
- **Données** : Interdiction des jointures SQL entre modules (complexifie les requêtes).
- **Setup** : Plus de temps de conception au démarrage.
- **Déploiement** : Reste une application unique (on déploie tout, tout le temps).

Exemple : Node JS (monolithique modulaire)



Exemple

```
app.js

const express = require("express");
const itemsRoutes = require("../src/modules/items/items.routes");
const playersRoutes = require("../src/modules/players/players.routes");

const app = express();
app.use(express.json());

app.use("/items", itemsRoutes);
app.use("/", playersRoutes);

app.listen(3000, () => {
  console.log("Server is running on port 3000");
});
```

```
src/shared/db.js

const Database = require("better-sqlite3");
const path = require("path");

const dbPath = path.join(__dirname, "../../database.db");
const db = new Database(dbPath);

db.exec(`
  CREATE TABLE IF NOT EXISTS players (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    pseudo TEXT UNIQUE,
    gems INTEGER DEFAULT 0
  );

  CREATE TABLE IF NOT EXISTS items (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    price INTEGER,
    stock INTEGER
  );
`);

//
const itemsCount = db.prepare('SELECT COUNT(*) as count FROM items').get().count;
if (itemsCount === 0) {
  const insertItem = db.prepare('INSERT INTO items (name, price, stock) VALUES (?, ?, ?)');
  insertItem.run('Épée en bois', 100, 10);
  insertItem.run('Bouclier basique', 150, 5);
  insertItem.run('Potion de soin', 50, 20);
}

const playersCount = db.prepare('SELECT COUNT(*) as count FROM players').get().count;
if (playersCount === 0) {
  const insertPlayer = db.prepare('INSERT INTO players (pseudo, gems) VALUES (?, ?)');
  insertPlayer.run('Joueur1', 200);
  insertPlayer.run('Joueur2', 50);
}

module.exports = db;
```

Exemple suite

```
src/players/players.routes.js

const express = require("express");
const router = express.Router();
const playersController = require("./players.controller");

router.post("/buy", playersController.buyItem);
router.post("/more-gemmes", playersController.addGems);

module.exports = router;
```

```
src/players/players.controller.js

const playersService = require("../players.service");

exports.buyItem = (req, res) => {
  const { playerId, itemId } = req.body;

  if (!playerId || !itemId) {
    return res.status(400).json({ error: "playerId et itemId sont requis" });
  }

  try {
    const result = playersService.buyItem(playerId, itemId);
    res.json(result);
  } catch (error) {
    if (error.message === "Joueur non trouvé" || error.message === "Objet non trouvé") {
      return res.status(404).json({ error: error.message });
    }
    res.status(400).json({ error: error.message });
  }
};

exports.addGems = (req, res) => {
  const { playerId, amount } = req.body;

  if (!playerId || amount === undefined) {
    return res.status(400).json({ error: "playerId et amount sont requis" });
  }

  if (amount <= 0) {
    return res.status(400).json({ error: "Le montant doit être supérieur à 0" });
  }

  try {
    const result = playersService.addGems(playerId, amount);
    res.json(result);
  } catch (error) {
    if (error.message === "Joueur non trouvé") {
      return res.status(404).json({ error: error.message });
    }
    res.status(500).json({ error: error.message });
  }
};
```

Exemple suite

```
src/players/players.service.js - 1

const db = require("../shared/db");

exports.buyItem = (playerId, itemId) => {
  const processPurchase = db.transaction((pId, iId) => {
    const player = db.prepare('SELECT * FROM players WHERE id = ?').get(pId);
    const item = db.prepare('SELECT * FROM items WHERE id = ?').get(iId);

    if (!player) throw new Error("Joueur non trouvé");
    if (!item) throw new Error("Objet non trouvé");

    if (item.stock <= 0) throw new Error("Objet en rupture de stock");
    if (player.gems < item.price) throw new Error("Fonds insuffisants");

    db.prepare('UPDATE items SET stock = stock - 1 WHERE id = ?').run(iId);
    db.prepare('UPDATE players SET gems = gems - ? WHERE id = ?').run(item.price, pId);

    return {
      success: true,
      message: "Achat réussi",
      remainingGems: player.gems - item.price,
      remainingStock: item.stock - 1
    };
  });

  return processPurchase(playerId, itemId);
};
```

```
src/players/players.service.js - 2

exports.addGems = (playerId, amount) => {
  const info = db.prepare('UPDATE players SET gems = gems + ? WHERE id = ?').run(amount,
  playerId);
  if (info.changes === 0) {
    throw new Error("Joueur non trouvé");
  }

  const player = db.prepare('SELECT * FROM players WHERE id = ?').get(playerId);
  return {
    success: true,
    message: `${amount} gemmes ajoutées`,
    newBalance: player.gems
  };
};
```

Exercise

Ex 2 - Monolithique modulaire

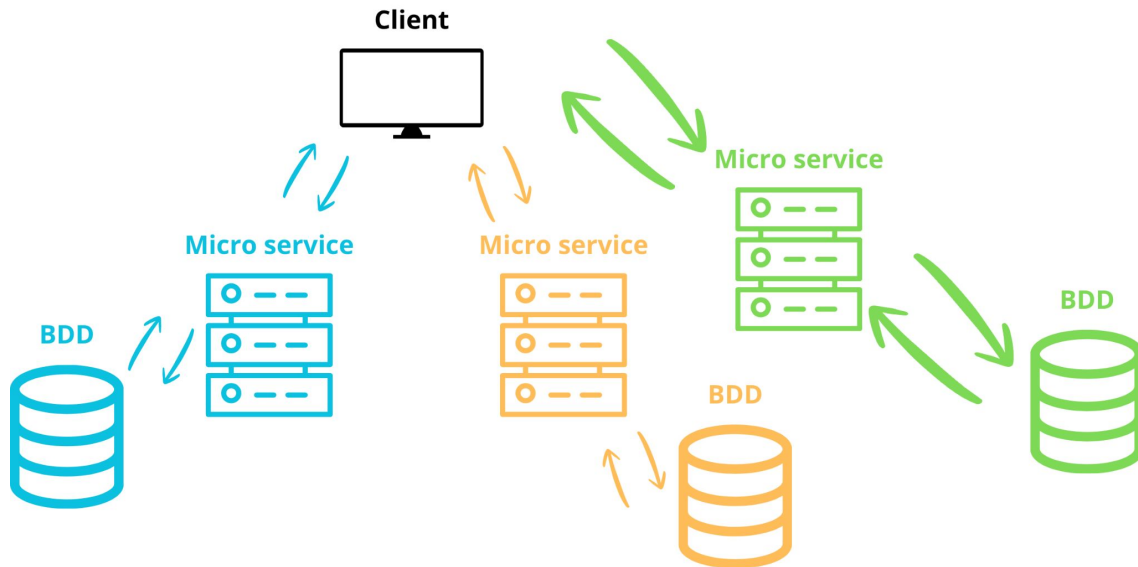
- Reprendre l'exercice précédent pour basculer vers une architecture plus modulaire.
- Contraintes de structure :
 - Séparation stricte : Interdiction d'avoir de la logique métier dans les routes.
 - Dossier services : Contient uniquement la logique pure (calculs, vérification solde/stock). Pas d'objets req ou res.
 - Dossier controllers : Gère uniquement les entrées/sorties HTTP. Il appelle le service et renvoie les statuts (200, 400, 404).
 - Dossier routes : Définit les points d'entrée et les lie aux contrôleurs.
 - Fichier app.js : Ne sert plus qu'à configurer Express et importer les routes.

Les micro-services



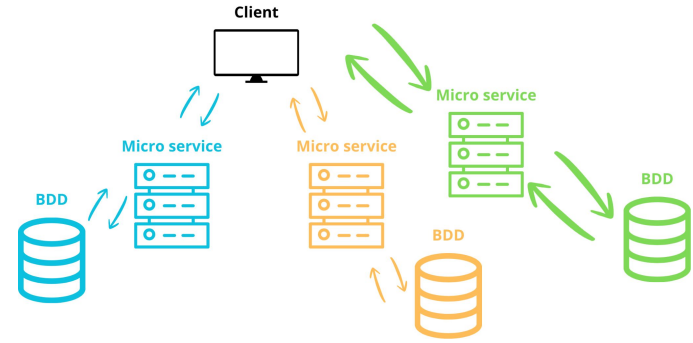
Les micro-services

- L'architecture en microservices découpe l'application en une **suite de petits services indépendants**



Caractéristiques des micro services

- Chaque service peut être développé, déployé et mis à l'échelle sans impacter les autres : **indépendance**
- Chaque service répond à une seule fonction métier : **spécialisation**
- Chaque service possède souvent sa propre base de données : **décentralisation**



Pourquoi utiliser les micro services ?

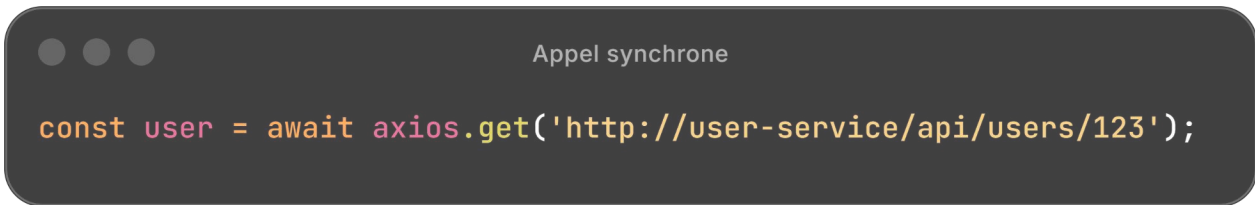
- Chaque micro service peut être écrit dans une stack différente
- Si un service est plus sollicité que les autres, on peut augmenter les ressources de celui ci spécifiquement
- Un service down ne veut pas dire que rien ne fonctionne
- Les équipes peuvent se séparer les tâches parallèlement plus facilement

Gestion des données

- Chaque micro service est **maître de ses données**
- Interdiction d'accéder directement à la BDD d'un autre service
- Exemple :
 - Service d'auth : MongoDB pour les users
 - Service paiement : PostgreSQL pour les transactions

Communication synchrone

- Un service appelle directement un autre et attend sa réponse
- Utilisation de requêtes HTTP (GET, POST) entre les services
- Exemple Express avec axios

A dark-themed terminal window with three window control buttons (red, yellow, green) in the top-left corner. The title bar text is "Appel synchrone". The code inside is a single line: `const user = await axios.get('http://user-service/api/users/123');`

```
Appel synchrone

const user = await axios.get('http://user-service/api/users/123');
```

Si le service appelé est lent, tout le système ralentit

Les défis du synchrone

- Si service A dépend de service B pour répondre, et que B est en panne, A tombe aussi. Si B est lent, A devient lent...
- Utiliser un **circuit breaker** :
 - Si B ne répond plus, A renvoie immédiatement une erreur
 - Permet de ne pas bloquer ses ressources à attendre
- Avec Node, on peut utiliser le package **opossum**

Circuit breaker, opossum

```
opossum node.js

const CircuitBreaker = require('opossum');

function asyncFunctionThatCouldFail(x, y) {
  return new Promise((resolve, reject) => {
    // Do something, maybe on the network or a disk
  });
}

const options = {
  timeout: 3000, // If our function takes longer than 3 seconds, trigger a failure
  errorThresholdPercentage: 50, // When 50% of requests fail, trip the circuit
  resetTimeout: 30000 // After 30 seconds, try again.
};

const breaker = new CircuitBreaker(asyncFunctionThatCouldFail, options);

breaker.fire(x, y)
  .then(console.log)
  .catch(console.error);
```

Exemple (items-service)

```
app.js - 1

const express = require("express");
const db = require("../database");

const app = express();
app.use(express.json());

// Récupérer tous les items
app.get("/items", (req, res) => {
  try {
    const items = db.prepare('SELECT * FROM items').all();
    res.json(items);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Récupérer un item spécifique
app.get("/items/:id", (req, res) => {
  try {
    const item = db.prepare('SELECT * FROM items WHERE id = ?').get(req.params.id);
    if (!item) {
      return res.status(404).json({ error: "Objet non trouvé" });
    }
    res.json(item);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

```
app.js - 2

// Décrémenter le stock d'un item (appelé par le service Players)
app.post("/items/:id/decrement", (req, res) => {
  try {
    const item = db.prepare('SELECT * FROM items WHERE id = ?').get(req.params.id);
    if (!item) {
      return res.status(404).json({ error: "Objet non trouvé" });
    }
    if (item.stock <= 0) {
      return res.status(400).json({ error: "Objet en rupture de stock" });
    }

    db.prepare('UPDATE items SET stock = stock - 1 WHERE id = ?').run(req.params.id);

    // Renvoyer l'item mis à jour
    const updatedItem = db.prepare('SELECT * FROM items WHERE id = ?').get(req.params.id);
    res.json({ success: true, item: updatedItem });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

const PORT = 3002;
app.listen(PORT, () => {
  console.log(`[Items Service] running on port ${PORT}`);
});
```


Exemple (players-service)

```
app.js - 1

const express = require("express");
const axios = require("axios");
const db = require("../database");
const CircuitBreaker = require("opossum");

const app = express();
app.use(express.json());

const ITEMS_SERVICE_URL = "http://localhost:3002";

const circuitBreakerOptions = {
  timeout: 3000, // Si la requête prend plus de 3s, on considère que c'est un échec
  errorThresholdPercentage: 50, // Ouvre le circuit après 50% d'échecs
  resetTimeout: 10000 // Attendre 10s avant de retenter (demi-ouvert)
};
```

```
app.js - 2

// On définit les fonctions qui seront appelées via le Circuit Breaker
const fetchItem = async (itemId) => {
  return await axios.get(`${ITEMS_SERVICE_URL}/items/${itemId}`);
};

const decrementStock = async (itemId) => {
  return await axios.post(`${ITEMS_SERVICE_URL}/items/${itemId}/decrement`);
};

const fetchItemBreaker = new CircuitBreaker(fetchItem, circuitBreakerOptions);
const decrementStockBreaker = new CircuitBreaker(decrementStock, circuitBreakerOptions);

// On peut ajouter des logs en option
fetchItemBreaker.on('open', () => console.log('[CircuitBreaker] fetchItem: OPEN'));
fetchItemBreaker.on('halfOpen', () => console.log('[CircuitBreaker] fetchItem: HALF-OPEN'));
fetchItemBreaker.on('close', () => console.log('[CircuitBreaker] fetchItem: CLOSED'));

decrementStockBreaker.on('open', () => console.log('[CircuitBreaker] decrementStock: OPEN'));
decrementStockBreaker.on('halfOpen', () => console.log('[CircuitBreaker] decrementStock: HALF-OPEN'));
decrementStockBreaker.on('close', () => console.log('[CircuitBreaker] decrementStock: CLOSED'));
```

Exemple (players-service)

```
app.js - 3

app.post("/more-gemmes", (req, res) => {
  const { playerId, amount } = req.body;

  if (!playerId || amount === undefined) {
    return res.status(400).json({ error: "playerId et amount sont requis" });
  }

  if (amount <= 0) {
    return res.status(400).json({ error: "Le montant doit être supérieur à 0" });
  }

  try {
    const info = db.prepare('UPDATE players SET gems = gems + ? WHERE id = ?').run(amount, playerId);
    if (info.changes === 0) {
      return res.status(404).json({ error: "Joueur non trouvé" });
    }

    const player = db.prepare('SELECT * FROM players WHERE id = ?').get(playerId);
    res.json({ success: true, message: `${amount} gemmes ajoutées`, newBalance: player.gems });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

```
app.js - 4

app.post("/buy", async (req, res) => {
  const { playerId, itemId } = req.body;
  if (!playerId || !itemId) { return res.status(400).json({ error: "playerId et itemId sont requis" }); }

  try {
    // 1. Vérifier le joueur localement
    const player = db.prepare('SELECT * FROM players WHERE id = ?').get(playerId);
    if (!player) {
      return res.status(404).json({ error: "Joueur non trouvé" });
    }

    // 2. Vérifier l'objet via Items Service (Appel HTTP GET Synchrone via Circuit Breaker)
    let itemResponse;
    try {
      itemResponse = await fetchItemBreaker.fire(itemId);
    } catch (err) {
      if (err.response && err.response.status === 404) {
        return res.status(404).json({ error: "Objet non trouvé (Items Service)" });
      }
      throw new Error("Erreur de communication avec le Items Service : " + err.message);
    }

    const item = itemResponse.data;

    // 3. Vérifier la logique métier (stock et fonds)
    if (item.stock <= 0) { return res.status(400).json({ error: "Objet en rupture de stock" }); }
    if (player.gems < item.price) { return res.status(400).json({ error: "Fonds insuffisants" }); }

    // 4. Décrémenter le stock via Items Service (Appel HTTP POST Synchrone via Circuit Breaker)
    try { await decrementStockBreaker.fire(itemId); }
    catch (err) {
      return res.status(400).json({ error: "Impossible de décrémenter le stock (Items Service) : " + err.message });
    }

    // 5. Décrémenter les gemmes du joueur localement
    db.prepare('UPDATE players SET gems = gems - ? WHERE id = ?').run(item.price, playerId);

    // 6. Renvoyer la réponse de succès
    const updatedPlayer = db.prepare('SELECT * FROM players WHERE id = ?').get(playerId);

    res.json({ success: true, message: "Achat réussi", remainingGems: updatedPlayer.gems, purchasedItem: item.name });

  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

Exercise

Ex 3 - Micro services (synchrone)

- Le service qui gère l'argent des étudiants est désormais séparé du service qui gère la nourriture
- wallet-service (Port 3001) :
 - Gère la base de données des utilisateurs et leurs soldes.
 - POST /debit : Reçoit un userId et un amount. Vérifie si le solde est suffisant et déduit l'argent.
 - POST /add : Ajoute de l'argent.
- menu-service (Port 3002) :
 - Gère le stock des plats.
 - GET /menu : Affiche les plats.
 - POST /order : Vérifie si le produit est en stock, Appelle le wallet-service pour tenter de débiter l'étudiant. Si le débit réussit, décrémente le stock et confirme la commande. Si le débit échoue, renvoie une erreur sans toucher au stock.
- Pour le moment, l'incohérence dans /order en cas de débit puis crash du service local est acceptée. Nous verrons comment le gérer plus tard

Communication asynchrone

- Les services communiquent via des messages sans attendre de réponse immédiate
- Un service publie un évènement, les autres s'y abonnent
- Utilisation d'un **message broker** (RabbitMQ, Kafka)



Communication asynchrone

```
// Publication d'un événement après un achat  
channel.sendToQueue('order_placed', Buffer.from(JSON.stringify(orderData)));
```

Qu'est-ce qu'un Message Broker ?

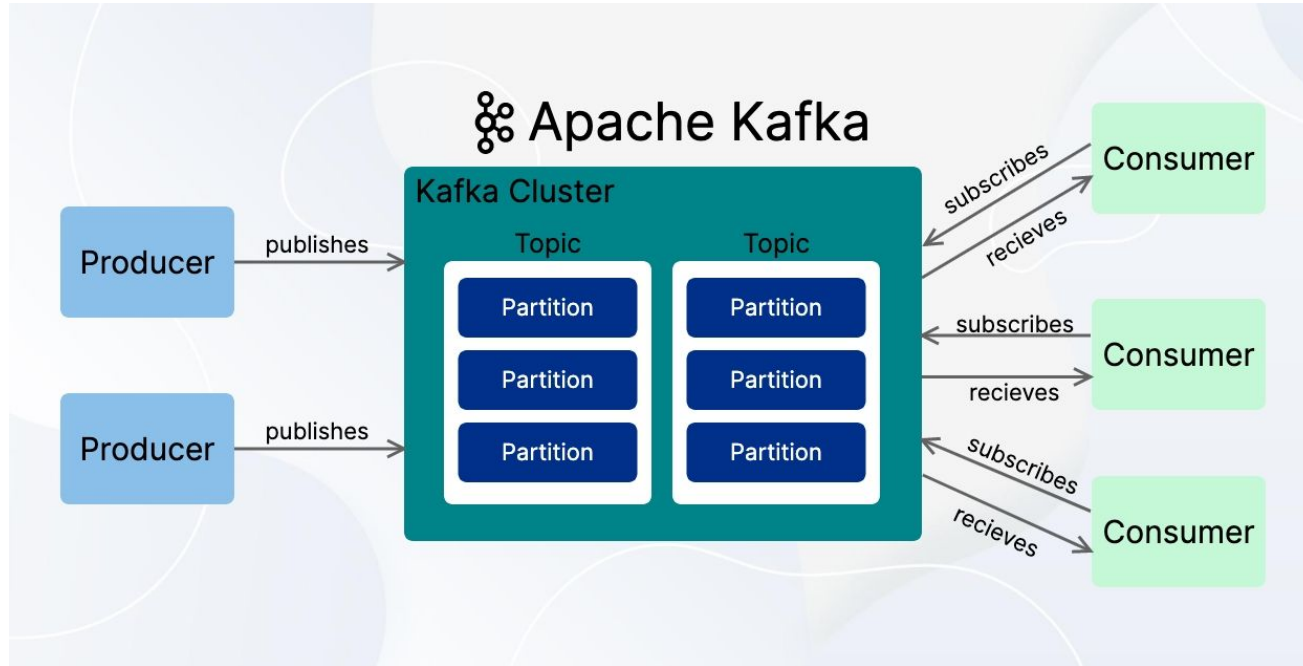
- Logiciel qui permet à différents systèmes de communiquer entre eux
- Reçoit, stocke et transmet les messages
- L'émetteurs de le récepteur n'ont pas besoin d'être connectés en même temps
- Garantit que les données ne sont pas perdues
- Permet au système d'envoyer une information et de continuer son travail sans attendre de réponse immédiate

Focus sur Apache Kafka

- Gestion de flux de données massifs avec une faible latence
- Stocke les messages comme une suite chronologique
- Kafka conserve les données pendant une durée définie (relecture)



Kafka en image



Kafka : mots clés

- **Producer** : Crée et envoie des messages vers un ou plusieurs topics.
- **Consumer** : S'abonne à un ou plusieurs topics pour lire et traiter les messages disponibles.
- **Broker** : Serveur au sein d'un cluster Kafka chargé de stocker les données et de servir les requêtes des clients.
- **Cluster** : Plusieurs Brokers travaillant ensemble pour assurer la haute disponibilité et la répartition de la charge.
- **Topic** : Catégorie ou nom de flux spécifique utilisé pour organiser les messages (similaire à une table en base de données).
- **Partition** : Subdivision d'un topic permettant de fragmenter les données pour les stocker sur différents brokers et paralléliser la lecture.

Kafka : scénario catastrophe

- Une action peut nécessiter deux écritures à deux endroits différents (ex : deux services interdépendants donc deux bases de données)
- Un crash peut par exemple intervenir après la première séquence d'écriture sur l'un des deux services
- Impossible d'utiliser les transactions pour assurer l'intégrité
- On utilisera le Saga Pattern

Saga Pattern : transactions distribuées

- Une suite de transactions locales dans chaque micro-service.
- Si une étape échoue, on lance des actions de compensation (annulation).
- **Garantit la cohérence** des données sans jointure SQL.
- Deux types :
 - Chorégraphie : Les services s'envoient des messages entre eux (via Kafka) sans chef d'orchestre
 - Orchestration : Un service central dit à chaque quoi faire

Exemple (config)

```
kafka.js

// config kafka
// shared/kafka.js par exemple
const { Kafka } = require('kafkajs');

const kafka = new Kafka({
  clientId: 'monoli-app',
  brokers: ['localhost:9092']
});

module.exports = kafka;
```

```
docker-compose.yml

version: '3'
services:
  kafka:
    image: confluentinc/cp-kafka:7.6.0
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
        'CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT'
      KAFKA_ADVERTISED_LISTENERS: 'PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092'
      KAFKA_PROCESS_ROLES: 'broker,controller'
      KAFKA_CONTROLLER_QUORUM_VOTERS: '1@kafka:29093'
      KAFKA_LISTENERS:
        'PLAINTEXT://0.0.0.0:29092,CONTROLLER://0.0.0.0:29093,PLAINTEXT_HOST://0.0.0.0:9092'
      KAFKA_INTER_BROKER_LISTENER_NAME: 'PLAINTEXT'
      KAFKA_CONTROLLER_LISTENER_NAMES: 'CONTROLLER'
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: 'true'
      CLUSTER_ID: 'Mku30EVbNTcwNTJENDM2Qk'
```

Exemple (items-service)

```
app.js

const express = require("express");
const routes = require("./routes");
const { initKafka } = require("./kafka");

const app = express();
app.use(express.json());

app.use("/", routes);

const PORT = 3001;
app.listen(PORT, async () => {
  await initKafka();
  console.log(`[Players Service] running on port ${PORT} (Async/Kafka)`);
});
```

```
database.js

const Database = require("better-sqlite3");
const path = require("path");

const dbPath = path.join(__dirname, "players.db");
const db = new Database(dbPath);

db.exec(`
  CREATE TABLE IF NOT EXISTS players (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    pseudo TEXT UNIQUE,
    gems INTEGER DEFAULT 0
  );

  CREATE TABLE IF NOT EXISTS orders (
    id TEXT PRIMARY KEY,
    playerId INTEGER,
    itemId INTEGER,
    status TEXT
  );
`);

const playersCount = db.prepare('SELECT COUNT(*) as count FROM players').get().count;
if (playersCount === 0) {
  const insertPlayer = db.prepare('INSERT INTO players (pseudo, gems) VALUES (?, ?)');
  insertPlayer.run('Joueur1', 200);
  insertPlayer.run('Joueur2', 50);
}

module.exports = db;
```

Exemple (items-service, kafka)

```
kafka.js 1/2

const kafka = require("../shared/kafka");
const { Partitioners } = require('kafkajs');
const db = require("../database");

const producer = kafka.producer({ createPartitioner: Partitioners.LegacyPartitioner });
const consumer = kafka.consumer({ groupId: 'items-group' });

async function initKafka() {
  const admin = kafka.admin();
  await admin.connect();
  await admin.createTopics({
    topics: [
      { topic: 'order.created' },
      { topic: 'order.item_reserved' },
      { topic: 'order.item_failed' },
      { topic: 'order.payment_completed' },
      { topic: 'order.payment_failed' }
    ]
  });
  await admin.disconnect();

  await producer.connect();
  await consumer.connect();

  await consumer.subscribe({ topic: 'order.created', fromBeginning: false });
  await consumer.subscribe({ topic: 'order.payment_failed', fromBeginning: false });

  await consumer.run({
    eachMessage: async ({ topic, partition, message }) => {
      const data = JSON.parse(message.value.toString());
      console.log('[Items Service] Reçu event: ${topic}', data);
    }
  });
}
```

```
kafka.js 2/2

if (topic === 'order.created') {
  const { orderId, playerId, itemId } = data;

  // Utilisation d'une transaction pour vérifier et réserver le stock
  const reserveItem = db.transaction((id) => {
    const item = db.prepare('SELECT * FROM items WHERE id = ?').get(id);
    if (!item) throw new Error('ITEM_NOT_FOUND');
    if (item.stock <= 0) throw new Error('OUT_OF_STOCK');

    db.prepare('UPDATE items SET stock = stock - 1 WHERE id = ?').run(id);
    return item;
  });

  try {
    const item = reserveItem(itemId);
    await producer.send({
      topic: 'order.item_reserved',
      messages: [{ value: JSON.stringify({ orderId, playerId, itemId, price: item.price }) }]
    });
    console.log('[Items Service] Réserve de l'objet ${itemId} réussie pour l'ordre ${orderId}.');
  } catch (error) {
    await producer.send({
      topic: 'order.item_failed',
      messages: [{ value: JSON.stringify({ orderId, reason: error.message }) }]
    });
    console.log('[Items Service] Réserve échouée pour l'ordre ${orderId}: ${error.message}.');
  }
}

if (topic === 'order.payment_failed') {
  // Rollback (Saga Compensation) : le joueur n'avait pas assez d'argent, on remet le stock
  const { itemId, orderId } = data;
  db.prepare('UPDATE items SET stock = stock + 1 WHERE id = ?').run(itemId);
  console.log('[Items Service] Rollback: objet ${itemId} restocké suite à l'échec de paiement pour ${orderId}.');
}

module.exports = { initKafka, producer };
```

Exemple (items-service, routes)

```
routes.js

const express = require("express");
const db = require("../database");

const router = express.Router();

router.get("/items", (req, res) => {
  const items = db.prepare('SELECT * FROM items').all();
  res.json(items);
});

module.exports = router;
```

Exemple (players-service)

```
app.js

const express = require("express");
const routes = require("./routes");
const { initKafka } = require("./kafka");

const app = express();
app.use(express.json());

app.use("/", routes);

const PORT = 3002;
app.listen(PORT, async () => {
  await initKafka();
  console.log(`[Items Service] running on port ${PORT} (Async/Kafka)`);
});
```

```
database.js

const Database = require("better-sqlite3");
const path = require("path");

const dbPath = path.join(__dirname, "items.db");
const db = new Database(dbPath);

db.exec(`
  CREATE TABLE IF NOT EXISTS items (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    price INTEGER,
    stock INTEGER
  );
`);

const itemsCount = db.prepare('SELECT COUNT(*) as count FROM items').get().count;
if (itemsCount === 0) {
  const insertItem = db.prepare('INSERT INTO items (name, price, stock) VALUES (?, ?, ?)');
  insertItem.run('Épée en bois', 100, 10);
  insertItem.run('Bouclier basique', 150, 5);
  insertItem.run('Potion de soin', 50, 20);
}

module.exports = db;
```


Exemple (players-service, kafka)

```
kafka.js 1/2

const kafka = require("../shared/kafka");
const { Partitioners } = require('kafkajs');
const db = require("../database");

const producer = kafka.producer({ createPartitioner: Partitioners.LegacyPartitioner });
const consumer = kafka.consumer({ groupId: 'players-group' });

async function initKafka() {
  const admin = kafka.admin();
  await admin.connect();
  await admin.createTopics({
    topics: [
      { topic: 'order.created' },
      { topic: 'order.item_reserved' },
      { topic: 'order.item_failed' },
      { topic: 'order.payment_completed' },
      { topic: 'order.payment_failed' }
    ]
  });
  await admin.disconnect();

  await producer.connect();
  await consumer.connect();

  await consumer.subscribe({ topic: 'order.item_reserved', fromBeginning: false });
  await consumer.subscribe({ topic: 'order.item_failed', fromBeginning: false });

  await consumer.run({
    eachMessage: async ({ topic, partition, message }) => {
      const data = JSON.parse(message.value.toString());
      console.log(`[Players Service] Reçu event: ${topic}`, data);
    }
  });
}
```

```
kafka.js 2/2

if (topic === 'order.item_reserved') {
  const { orderId, playerId, itemId, price } = data;
  const player = db.prepare('SELECT * FROM players WHERE id = ?').get(playerId);

  if (player.gems >= price) {
    // Paiement réussi
    db.prepare('UPDATE players SET gems = gems - ? WHERE id = ?').run(price, playerId);
    db.prepare('UPDATE orders SET status = ? WHERE id = ?').run('SUCCESS', orderId);

    await producer.send({
      topic: 'order.payment_completed',
      messages: [{ value: JSON.stringify({ orderId, itemId }) }]
    });
    console.log(`[Players Service] Achat ${orderId} validé avec succès.`);
  } else {
    // Fonds insuffisants
    db.prepare('UPDATE orders SET status = ? WHERE id = ?').run('FAILED_INSUFFICIENT_FUNDS', orderId);
    await producer.send({
      topic: 'order.payment_failed',
      messages: [{ value: JSON.stringify({ orderId, itemId, reason: 'Fonds insuffisants' }) }]
    });
    console.log(`[Players Service] Achat ${orderId} échoué : Fonds insuffisants.`);
  }
}

if (topic === 'order.item_failed') {
  const { orderId, reason } = data;
  db.prepare('UPDATE orders SET status = ? WHERE id = ?').run('FAILED_${reason}', orderId);
  console.log(`[Players Service] Achat ${orderId} échoué : ${reason}`);
}

module.exports = { initKafka, producer };
```

Exemple (players-service, route)

```
routes.js 1/2

const express = require("express");
const { v4: uuidv4 } = require("uuid");
const db = require("../database");
const { producer } = require("../kafka");

const router = express.Router();

router.post("/more-gemmes", (req, res) => {
  const { playerId, amount } = req.body;
  if (!playerId || amount === undefined) {
    return res.status(400).json({ error: "playerId et amount sont requis" });
  }

  const info = db.prepare('UPDATE players SET gems = gems + ? WHERE id = ?').run(amount, playerId);
  if (info.changes === 0) return res.status(404).json({ error: "Joueur non trouvé" });

  const player = db.prepare('SELECT * FROM players WHERE id = ?').get(playerId);
  res.json({ success: true, message: `${amount} gemmes ajoutées`, newBalance: player.gems });
});

// Achat asynchrone (Choreography avec Kafka)
router.post("/buy", async (req, res) => {
  const { playerId, itemId } = req.body;

  const player = db.prepare('SELECT * FROM players WHERE id = ?').get(playerId);
  if (!player) return res.status(404).json({ error: "Joueur non trouvé" });

  const orderId = uuidv4();
  db.prepare('INSERT INTO orders (id, playerId, itemId, status) VALUES (?, ?, ?, ?)').run(orderId, playerId, itemId, 'PENDING');
});
```

```
routes.js 2/2

// Émission de l'événement vers Kafka
await producer.send({
  topic: 'order.created',
  messages: [{ value: JSON.stringify({ orderId, playerId, itemId }) }]
});

// Réponse immédiate au client (le reste se fait en asynchrone)
res.status(202).json({
  message: "Demande d'achat prise en compte.",
  orderId,
  status: 'PENDING',
  checkUrl: `/orders/${orderId}`
});

router.get("/orders/:id", (req, res) => {
  const order = db.prepare('SELECT * FROM orders WHERE id = ?').get(req.params.id);
  if (!order) return res.status(404).json({ error: "Commande non trouvée" });
  res.json(order);
});

module.exports = router;
```

Ex 4 - Micro services (Kafka)

- Reprendre l'exercice précédent et modifier l'approche : on utilisera kafka pour une communication asynchrone.
- Vous pourrez reprendre l'exemple précédent comme model pour la logique kafka

API Gateway

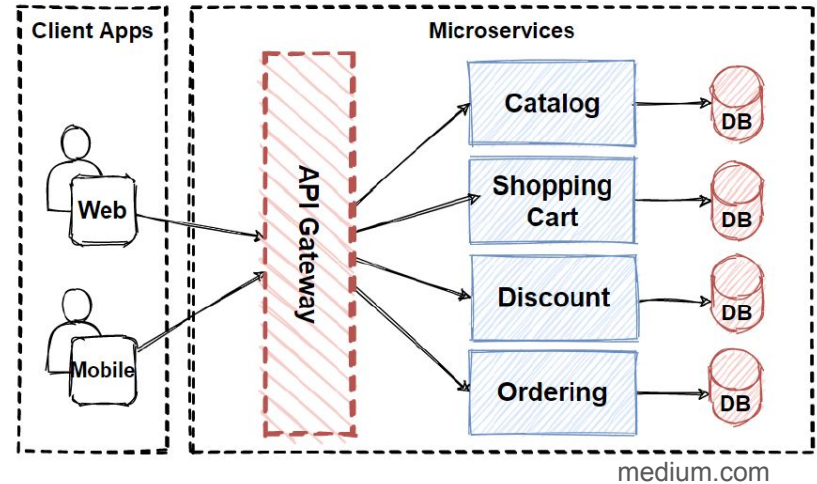
- Fournir un **point d'entrée unique** pour le client qui redirige les requêtes
- Reçoit la requête client et redirige vers le bon service

Exemple proxy express

```
app.use('/users', proxy('http://user-service:3000'));  
app.use('/orders', proxy('http://order-service:3001'));
```

API Gateway

- Au-delà du routage, permet des plusieurs choses :
 - Sécurité via auth centralisée
 - Rate limiting pour anti spam des utilisateurs
 - Observabilité (cf suite cours)



Exemple

```
nouveauDossier/app.js

const express = require('express');
// Standard express : http-proxy-middleware
const { createProxyMiddleware } = require('http-proxy-middleware');

const app = express();
const PORT = 3000;

// Routage vers le Players Service
app.use(
  ['/more-gemmes', '/buy', '/orders'],
  createProxyMiddleware({
    target: 'http://localhost:3001',
    changeOrigin: true
  })
);

// Routage vers le Items Service
app.use(
  '/items',
  createProxyMiddleware({
    target: 'http://localhost:3002',
    changeOrigin: true
  })
);

// Accueil
app.get('/', (req, res) => {
  res.send('API Gateway en ligne');
});

app.listen(PORT, () => {
  console.log(`[API Gateway] running on http://localhost:${PORT}`);
});
```

Exercise

Ex 4 - Micro services et API Gateway

- Reprendre l'exercice précédent
- Ajouter une API Gateway pour gérer le routage vers les bons services
- Bonus : Essayer d'implémenter une logique d'authentification centralisée avec un JWT

Observabilité et Logs

- Comment savoir ce qu'il se passe quand une erreur traverse 5 services ?
 - **Tracé distribué** : on ajoute un ID à chaque requête
 - **Logs centralisés** : on regroupe les logs dans un endroit (ELK stack)
 - **Health checks** : chaque service doit exposer une route /health



Le tracé distribué

- Suivre le cheminement d'une requête à travers tous les services
- On génère un Correlation-ID unique dès l'entrée de la requête.
- Cet ID est transmis dans les headers HTTP de service en service.

Logs centralisés

- Regrouper tous les fichiers de logs pour ne pas avoir à se connecter sur chaque serveur
- Elasticsearch : Moteur de recherche pour stocker les logs.



elasticsearch

Health checks et monitoring

- Permet de Surveiller la santé individuelle de chaque brique du système :
 - Le service est-il vivant ? Sinon on le redémarre
 - Le service est-il prêt à recevoir du trafic ? (BBD fonctionnelle etc)

Exercise

Ex 5 - Micro services et observabilité

- Reprendre l'exercice précédent, et ajouter :
 - Une route /health pour chaque service (connexion DB et kafka toujours active ?)
 - Elasticsearch pour la gestion des logs
 - Un "Correlation-ID" pour chaque requête

Qu'est ce qu'Open API

- Spécification standard pour décrire une API REST
- Aussi appelé Swagger
- Décrit notamment :
 - Endpoints
 - Paramètres
 - Réponses
 - Modèles
- Fichier généralement au format JSON



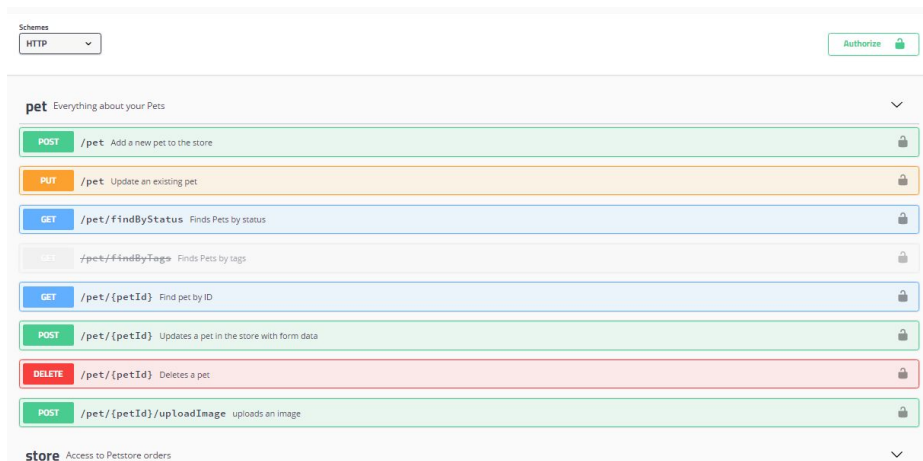
Pourquoi utiliser OpenAPI

- Documentation **claire** et **standardisée**, source de “vérité” de l’API
- Tester l’API directement depuis le navigateur
- Meilleure **collaboration front / back**



Swagger c'est quoi ?

- Ce sont les outils autour d'OpenAPI
- Nous utiliserons Swagger UI (interface web interactive)



Swagger pour Express.js

- Pour documenter une API Express, on installe les modules :
 - express
 - swagger-ui-express
 - swagger-jsdoc

Configuration swagger (swagger.js)

```
const swaggerJsdoc = require("swagger-jsdoc");

const options = {
  definition: {
    openapi: "3.0.0",
    info: {
      title: "Mon API Express",
      version: "1.0.0",
      description: "Documentation de mon API"
    },
  },
  apis: ["/routes/*.js"],
};

module.exports = swaggerJsdoc(options);
```

Intégration dans express

```
const express = require("express");
const swaggerUi = require("swagger-ui-express");
const swaggerSpec = require("./swagger");

const app = express();

app.use("/api-docs", swaggerUi.serve, swaggerUi.setup(swaggerSpec));

app.listen(3000, () => {
  console.log("API running on http://localhost:3000");
});
```

Documenter une route

```
/**
 * @swagger
 * /users:
 *   get:
 *     summary: Récupère la liste des utilisateurs
 *     responses:
 *       200:
 *         description: Succès
 */
router.get("/users", (req, res) => {
  res.json([]);
});
```

Bonnes pratiques

```
/**
 * @swagger
 * /users:
 *   get:
 *     summary: Récupère la liste des utilisateurs
 *     responses:
 *       200:
 *         description: Succès
 */
router.get("/users", (req, res) => {
  res.json([]);
});
```

Bonnes pratiques

- Documenter **toutes les routes**
- **Définir des schémas**/modèles pour les objets
- Garder la doc à jour avec le code

Ex 6 - Mise en place d'un swagger

- Reprendre l'exercice et générer une documentation OpenAPI pour chaque microservice.

TP évalué : Validation des acquis